

EOAT Command Center Restructure Phase Plan

Purpose

This document is the six-phase implementation plan for restructuring and improving the EOAT Command Center app.

It is meant to go with the larger architecture improvement document. The architecture document explains **what should change and why**. This phase plan explains **how to implement it in a controlled order** so Codex can work through the improvements without accidentally turning the app into a smoking pile of PySide widgets and Excel ghosts.

This plan intentionally keeps the work to **six total phases**.

This plan also intentionally excludes tag-category changes for now.

Overall Goals

The goal is to make the EOAT Command Center easier to maintain, easier to expand, and safer to modify.

The biggest problems being solved are:

1. Audit fields are defined in too many places.
2. Audit progress/completion rules are becoming too complicated for scattered one-off logic.
3. The Audit page is doing too much.
4. Workbook schema changes need safer, versioned migrations.
5. Cylinder fields need to be added cleanly.
6. Manual completion override needs to be truthful and audit-specific.
7. Press View, compatibility rows, audit progress, and reports need a shared relationship model.
8. Dashboard routing and event handling need to be more modular and resilient.

The expected final result is an app where future changes like “add a new EOAT field” require updating a clean schema and a small number of focused modules, instead of spelunking through half the repo with a flashlight and a prayer.

Phase Overview

Phase 1: Audit Field Registry and Schema Foundation

Create the central audit field registry so the app has one authoritative definition for each audit field.

Phase 2: Completion Policy Engine and Audit Coach Refactor

Move completion/progress rules into a dedicated testable engine.

Phase 3: Cylinder Fields and Manual Completion Override

Add the new Cylinder mini-section and the manual override feature using the new schema/completion architecture.

Phase 4: Audit Page Modularization

Split the large Audit page into smaller controllers and builders while preserving behavior.

Phase 5: Workbook Repository, Migrations, and Relationship Service

Separate workbook persistence from audit domain logic and create shared relationship logic for physical/compatibility rows.

Phase 6: Dashboard Routing, Event Bus Hardening, Tests, and Cleanup

Clean up dashboard routing, harden events, strengthen tests, and remove transitional duplication.

Phase 1: Audit Field Registry and Schema Foundation

Phase Objective

Create a centralized audit field registry that defines each audit field exactly once.

This becomes the foundation for UI generation, workbook headers, validation, dropdown values, default values, visibility rules, and progress policies.

Right now, audit field information is scattered across UI code, workbook schema, audit rules, validation, and coach logic. That means every new field has to be manually wired into multiple places. This phase starts fixing that.

Expected Result

At the end of Phase 1:

- The app has a new central audit schema module.
- Every current audit field is represented in the registry.
- Existing behavior remains unchanged.
- Existing tests still pass.
- Future fields can be added through the registry instead of scattered hardcoded lists.
- The existing UI does not need to be fully generated from the registry yet, but the registry should be ready for that.

New / Updated Files

Recommended new files:

```
core/audit/schema.py
tests/core/audit/test_schema.py
```

Likely updated files:

```
core/audit_entries.py
core/audit_field_rules.py
core/workbook_schema.py
app/pages/audit.py
data_templates/workbook_schema.json
```

The first phase should avoid deleting old constants. Keep old constants as wrappers or aliases so the app does not explode like a bargain-bin solenoid.

Implementation Details

1. Create `AuditFieldSpec`

Create a new dataclass in:

```
core/audit/schema.py
```

Recommended structure:

```

from dataclasses import dataclass
from typing import Literal

WidgetType = Literal[
    "line_edit",
    "text_edit",
    "combo",
    "checkbox",
]

StorageTarget = Literal[
    "eoa_master",
    "robot_info",
    "audit_metadata",
    "none",
]

ProgressPolicy = Literal[
    "normal",
    "excluded",
    "optional_group_trigger",
    "optional_group_dependent",
    "manual_override_metadata",
]

VisibilityRule = Literal[
    "always",
    "vacuum_tooling",
    "gripper_tooling",
    "sensor_detail",
    "electrical_detail",
    "quick_disconnect_detail",
    "compatible_row_hidden",
]

@dataclass(frozen=True)
class AuditFieldSpec:
    field_id: str
    label: str
    workbook_header: str
    section: str
    group: str
    widget_type: WidgetType
    storage_target: StorageTarget = "eoa_master"
    dropdown_values: tuple[str, ...] = ()
    editable_dropdown: bool = False
    include_blank: bool = True

```

```

default: str | None = None
numeric: bool = False
required_for_audited: bool = False
required_for_compatible: bool = False
important: bool = False
visibility_rule: VisibilityRule = "always"
progress_policy: ProgressPolicy = "normal"
optional_group: str | None = None
legacy_headers: tuple[str, ...] = ()
tags: tuple[str, ...] = ()
help_text: str = ""

```

2. Add all current audit fields

Every existing EOAT Inventory field should be represented.

Examples:

```

AuditFieldSpec(
    field_id="audit_id",
    label="Audit ID",
    workbook_header="Audit ID",
    section="Audit Header",
    group="Audit Identity",
    widget_type="line_edit",
    required_for_audited=True,
)

AuditFieldSpec(
    field_id="audit_date",
    label="Audit Date",
    workbook_header="Audit Date",
    section="Audit Header",
    group="Audit Identity",
    widget_type="line_edit",
    required_for_audited=True,
)

AuditFieldSpec(
    field_id="eoa_type",
    label="EOAT Type",
    workbook_header="EOAT Type",
    section="EOAT Type and Tooling",
    group="EOAT Classification",
    widget_type="combo",
    dropdown_values=(

```

```

        "Vacuum",
        "Mechanical / Gripper",
        "Hybrid",
        "Unknown / Needs Review",
        "Miscellaneous",
    ),
    required_for_audited=True,
)

```

3. Preserve legacy labels

For renamed fields, use stable internal IDs and legacy aliases.

Example:

```

AuditFieldSpec(
    field_id="number_of_parts_picked",
    label="Number of Parts Picked",
    workbook_header="Number of Parts Picked",
    section="EOAT Type and Tooling",
    group="Part Pickup",
    widget_type="line_edit",
    numeric=True,
    legacy_headers=("Number of Vacuum Cups",),
)

```

This prevents future renames from causing another Excel séance.

4. Add registry helper functions

Add functions like:

```

def all_audit_fields() -> tuple[AuditFieldSpec, ...]:
    ...

def field_by_id(field_id: str) -> AuditFieldSpec:
    ...

def field_by_header(header: str) -> AuditFieldSpec | None:
    ...

def fields_for_section(section: str) -> list[AuditFieldSpec]:
    ...

def fields_grouped_by_section() -> dict[str, dict[str, list[AuditFieldSpec]]]:

```

```

...

def expected_workbook_headers(storage_target: str = "eoa_master") -> list[str]:
    ...

def dropdown_values_for(field_id_or_header: str) -> tuple[str, ...]:
    ...

```

5. Keep existing constants temporarily

Do not immediately delete old lists like:

```

AUDIT_SECTIONS
AUDIT_DROPDOWNS
AUDIT_REQUIRED_FIELDS
AUDIT_IMPORTANT_FIELDS
TOOLING_COLUMN_ORDER

```

Instead, begin deriving new versions from the registry where low-risk.

For example:

```

def audit_sections_from_registry() -> dict[str, list[str]]:
    ...

```

Then tests can compare old and new structures before the UI is migrated.

Acceptance Criteria

Phase 1 is complete when:

- `core/audit/schema.py` exists.
 - Every current EOAT Inventory field is represented.
 - Each field has a stable `field_id`.
 - There are no duplicate field IDs.
 - There are no unexpected duplicate workbook headers.
 - Existing workbook headers can be generated from the registry.
 - Existing audit behavior remains unchanged.
 - Existing tests pass.
 - New registry tests pass.
-

Testing Requirements

Add tests for:

```
tests/core/audit/test_schema.py
```

Test cases:

1. Registry is not empty.
2. Every field has a non-empty `field_id`.
3. Every field has a non-empty label.
4. Every field has a non-empty workbook header unless storage target is `none`.
5. Field IDs are unique.
6. Workbook headers are unique per storage target.
7. Current EOAT Inventory schema headers are covered by the registry.
8. Legacy headers resolve to current fields.
9. Dropdown fields expose expected values.
10. Numeric fields are correctly marked.
11. Fields are grouped into expected sections.
12. Required fields are correctly marked.

Codex Instructions for Phase 1

Codex should implement Phase 1 without changing app behavior.

Codex should report:

- Files changed
- Registry fields added
- Any fields it could not confidently classify
- Tests added
- Test results
- Any remaining duplicated field definitions that will be handled in later phases

Phase 2: Completion Policy Engine and Audit Coach Refactor

Phase Objective

Create a dedicated completion/progress engine for audits.

Right now, audit coach completion is based mostly on whether a field applies and whether it has a meaningful value. That is not enough anymore.

The app now needs to understand:

- Fields excluded from completion
- Optional groups
- Unknown / Not Checked
- Not applicable
- Stale hidden conflicts
- Manual override
- Required vs important fields
- Field-level truth vs audit-level completion percentage

Because obviously “is this audit complete?” has become a philosophical question with dropdowns.

Expected Result

At the end of Phase 2:

- Completion logic lives in `core/audit/completion.py`.
- The audit coach uses the completion engine.
- Completion behavior is testable without launching the UI.
- Notes and other excluded fields can be ignored cleanly.
- Optional group support exists, ready for cylinders.
- Manual override support exists at the engine level, even if the UI button is added in Phase 3.
- Existing audit coach behavior remains mostly the same except for intentionally excluded fields.

New / Updated Files

Recommended new files:

```
core/audit/completion.py
tests/core/audit/test_completion.py
```

Likely updated files:

```
core/audit/coach.py
core/audit_field_rules.py
core/audit/schema.py
app/pages/audit_coach_panel.py
app/pages/audit.py
```

Implementation Details

1. Create completion state model

In:

```
core/audit/completion.py
```

Add states:

```
STATE_VERIFIED_COMPLETE = "verified_complete"  
STATE_MISSING = "missing"  
STATE_UNKNOWN_NOT_CHECKED = "unknown_not_checked"  
STATE_NOT_APPLICABLE = "not_applicable"  
STATE_FOLLOW_UP_NEEDED = "follow_up_needed"  
STATE_STALE_CONFLICT = "stale_conflict"  
STATE_EXCLUDED = "excluded"  
STATE_IGNORED_BY_OPTIONAL_GROUP = "ignored_by_optional_group"  
STATE_IGNORED_BY_MANUAL_OVERRIDE = "ignored_by_manual_override"
```

Create dataclasses:

```
@dataclass(frozen=True)  
class FieldCompletionStatus:  
    field_id: str  
    label: str  
    workbook_header: str  
    section: str  
    group: str  
    state: str  
    value: str  
    applies: bool  
    counts_toward_progress: bool  
    required: bool = False  
    important: bool = False  
    reason: str = ""  
  
@dataclass(frozen=True)  
class SectionCompletionStatus:  
    section: str  
    applicable_count: int  
    counted_count: int  
    verified_count: int
```

```

        missing_count: int
        ignored_count: int
        percent_complete: int
        fields: tuple[FieldCompletionStatus, ...]

@dataclass(frozen=True)
class AuditCompletionSummary:
    audit_id: str
    percent_complete: int
    complete_by_manual_override: bool
    verified_count: int
    counted_count: int
    missing_fields: tuple[str, ...]
    ignored_fields: tuple[str, ...]
    sections: tuple[SectionCompletionStatus, ...]
    explanation: str

```

2. Implement `calculate_audit_completion`

Suggested function:

```

def calculate_audit_completion(
    entry: Mapping[str, Any],
    *,
    field_specs: Iterable[AuditFieldSpec] | None = None,
    mode: str = "",
) -> AuditCompletionSummary:
    ...

```

It should:

1. Normalize field values into strings.
2. Check whether the audit has manual completion override.
3. Classify every field.
4. Group fields by section.
5. Count only fields that should count.
6. Return a truthful audit summary.

3. Implement excluded fields

Fields with this policy should not reduce completion:

```

progress_policy = "excluded"

```

Recommended excluded fields:

Notes
Tubing Routing Notes

Possibly also:

Photo Folder/Link

Only exclude Photo Folder/Link if the intended behavior is that photos may be optional. If Photos Taken is Yes, Photo Folder/Link may still deserve a warning.

4. Implement optional group support

The completion engine should understand this pattern:

```
optional_group_trigger  
optional_group_dependent
```

For an optional group:

- If no trigger field has a meaningful value, ignore all dependent fields.
- If a trigger field has a meaningful value, count the group normally.

This is needed for cylinders in Phase 3.

5. Implement manual override support

The completion engine should recognize:

Manual Completion Override = Yes

or equivalent truthy values.

When active:

- Audit-level percent complete is 100%.
- `complete_by_manual_override = True`.
- Missing fields should be listed as ignored, not verified.
- Field-level truth should remain intact.

Example explanation:

This audit is marked complete by manual override. Blank fields are ignored for completion percentage, but they were not field-verified.

6. Refactor audit coach to use completion engine

`core/audit/coach.py` should stop duplicating raw completion math where possible.

It can still provide coach-specific extras like:

- next best field
- guided fields
- warnings
- semantic findings
- stale hidden values
- follow-up actions

But the underlying field completion state should come from `completion.py`.

Acceptance Criteria

Phase 2 is complete when:

- `core/audit/completion.py` exists.
 - Completion can be calculated without UI.
 - Audit coach uses completion results.
 - Excluded fields do not reduce progress.
 - Optional group logic is implemented and tested.
 - Manual override logic is implemented and tested at the core level.
 - Existing coach UI still works.
 - Existing tests pass.
 - New completion tests pass.
-

Testing Requirements

Add tests in:

`tests/core/audit/test_completion.py`

Test cases:

1. Fully filled audit is 100%.
2. Missing required field reduces completion.

3. Missing important field reduces completion if it is part of counted fields.
 4. `Unknown / Not Checked` is not verified complete.
 5. `N/A` field is ignored if not applicable.
 6. Meaningful non-applicable field becomes stale conflict.
 7. Excluded fields do not reduce completion.
 8. Empty optional group does not reduce completion.
 9. Active optional group counts normally.
 10. Manual override returns 100%.
 11. Manual override does not mark missing fields as verified.
 12. Manual override lists ignored fields.
 13. Compatibility rows use compatible-row requirements.
 14. Physical audit rows use physical-audit requirements.
-

Codex Instructions for Phase 2

Codex should implement the completion engine and refactor the audit coach carefully.

Codex should avoid changing UI layout in this phase unless necessary.

Codex should report:

- Completion states added
 - Fields excluded from progress
 - How optional groups work
 - How manual override is represented internally
 - Audit coach changes
 - Tests added
 - Test results
-

Phase 3: Cylinder Fields and Manual Completion Override

Phase Objective

Add the actual user-facing features:

1. Cylinder mini-section
2. Manual completion override button

This phase uses the registry and completion engine from Phases 1 and 2.

This is where the app gets the new functionality the user actually asked for, instead of just receiving a lovely architecture skeleton wearing a hard hat.

Part A: Cylinder Mini-Section

Expected Result

At the end of this phase:

- `# of Cylinders` exists on the EOAT Audit form.
 - `Cylinder Type` exists on the EOAT Audit form.
 - The Cylinder section appears on the `EOAT Type and Tooling` tab.
 - The section never disappears.
 - `Cylinder Type` has dropdown values `Linear` and `Rotary`.
 - `Cylinder Type` defaults to `Linear`.
 - Empty cylinder fields do not reduce audit completion.
 - Filled `# of Cylinders` activates cylinder progress behavior.
 - Cylinder fields save and load correctly.
 - Existing audits without these fields still load.
-

Cylinder Field Definitions

Add to the audit field registry:

```
AuditFieldSpec(  
    field_id="cylinder_count",  
    label="# of Cylinders",  
    workbook_header="# of Cylinders",  
    section="EOAT Type and Tooling",  
    group="Cylinder Details",  
    widget_type="line_edit",  
    numeric=True,  
    visibility_rule="always",  
    progress_policy="optional_group_trigger",  
    optional_group="cylinders",  
)  
  
AuditFieldSpec(  
    field_id="cylinder_type",  
    label="Cylinder Type",  
    workbook_header="Cylinder Type",  
    section="EOAT Type and Tooling",
```

```

group="Cylinder Details",
widget_type="combo",
dropdown_values=("Linear", "Rotary"),
editable_dropdown=False,
include_blank=False,
default="Linear",
visibility_rule="always",
progress_policy="optional_group_dependent",
optional_group="cylinders",
)

```

UI Behavior

Add a group on the EOAT Type and Tooling tab:

Cylinder Details

Fields:

of Cylinders
Cylinder Type

Visibility:

Always visible

It should remain visible for:

- Vacuum
- Mechanical / Gripper
- Hybrid
- Miscellaneous
- Unknown / Needs Review
- Blank EOAT Type

Progress Behavior

Cylinder group should be optional.

Rules:


```
If # of Cylinders is blank:
    Ignore # of Cylinders for completion.
    Ignore Cylinder Type for completion.
    Do not let default Linear create fake completion.

If # of Cylinders has a value:
    Count # of Cylinders.
    Count Cylinder Type.
    If Cylinder Type is blank, default it to Linear.
```

The important trap: `Cylinder Type = Linear` should not by itself make the audit look more complete. Otherwise every EOAT gets a ghost cylinder, because apparently dropdown defaults are tiny liars.

Workbook Behavior

Add headers to EOAT Inventory:

```
# of Cylinders
Cylinder Type
```

Recommended position:

```
Number of Parts Picked
# of Grippers
Gripper Type
Gripper Model
Gripper Size
# of Cylinders
Cylinder Type
# of Cups
Cup Type/Material
Cup Diameter/Size
Vacuum Generator Type
```

Validation

Rules:

```
# of Cylinders:
    blank or non-negative integer
```

Cylinder Type:
blank, Linear, or Rotary

If # of Cylinders is filled and Cylinder Type is blank, normalize to:

Linear

Part B: Manual Completion Override

Expected Result

At the end of this phase:

- The Audit page has a manual override button.
- The user gets a confirmation popup before applying it.
- The override applies only to the current audit.
- The override is saved.
- The override persists after reload.
- The audit coach shows 100% completion by manual override.
- The app does not pretend every field was verified.
- Other audits are unaffected.

Button Label

Use:

Manual Override: Mark Audit Complete

Recommended Placement

Put it near the Audit Coach panel or in the audit action button row.

Best recommendation:

- Put a button inside or near the Audit Coach panel.
- This makes the purpose obvious because the override affects completion.

Confirmation Popup

Title:

Manual Completion Override

Message:

This will manually mark this audit as complete even though some fields may still be blank.

Blank fields currently remaining on this audit will be ignored by the audit coach for completion percentage purposes.

This only affects the current audit. It does not change global validation rules.

Are you sure you want to continue?

Buttons:

Cancel
Mark Complete

Default button:

Cancel

Metadata Fields

Add workbook metadata fields:

Manual Completion Override
Manual Completion Override Timestamp
Ignored Empty Fields At Override

Optional:

Manual Completion Override Note
Manual Completion Override User

Recommended first implementation:

Manual Completion Override
Manual Completion Override Timestamp
Ignored Empty Fields At Override

Save Behavior

When user confirms:

1. Calculate current completion.
2. Capture currently missing counted fields.
3. Set `Manual Completion Override` to `Yes`.
4. Set timestamp.
5. Save ignored field list.
6. Save audit.
7. Refresh audit coach.
8. Mark form baseline clean after successful save.
9. Show result message.

Result message example:

Manual completion override applied. This audit is treated as 100% complete, but blank fields were not field-verified.

Audit Coach Display

When override is active:

Show:

100% Complete by Manual Override

Also show warning/note:

This audit was manually marked complete. Some blank fields may have been ignored for completion percentage.

Manual Override Must Not

Do not:

- Set every blank field to fake values.
 - Change global required fields.
 - Change global progress policy.
 - Affect other audits.
 - Silently disappear when the audit is edited.
 - Block normal saving.
-

New / Updated Files

Likely updated:

```
core/audit/schema.py
core/audit/completion.py
core/audit_entries.py
core/audit_field_rules.py
data_templates/workbook_schema.json
app/pages/audit.py
app/pages/audit_coach_panel.py
```

Possible new:

```
core/audit/manual_override.py
tests/core/audit/test_cylinders.py
tests/core/audit/test_manual_override.py
tests/app/test_audit_cylinder_ui.py
tests/app/test_manual_override_ui.py
```

Acceptance Criteria

Phase 3 is complete when:

- Cylinder fields are visible and functional.
- Cylinder completion rules work.
- Cylinder fields save and load.
- Workbook migration adds cylinder columns.
- Manual override button exists.
- Confirmation popup works.

- Override saves and reloads.
 - Audit coach shows 100% by override.
 - Field-level truth remains available.
 - Existing save/load behavior is not broken.
 - Tests pass.
-

Phase 4: Audit Page Modularization

Phase Objective

Split the large Audit page into smaller modules/controllers.

The Audit page currently does too much. This phase makes the code easier to maintain by separating responsibilities.

This phase should preserve user-facing behavior. The goal is to move logic, not redesign the UI.

Expected Result

At the end of Phase 4:

- `AuditPage` is significantly smaller.
 - Form state logic is separate.
 - Widget creation is separate.
 - Form building is separate.
 - Draft logic is separate.
 - Visibility logic is separate.
 - Coach logic is separate.
 - Save/load logic is better isolated.
 - Existing behavior remains intact.
 - Tests still pass.
-

Proposed File Structure

Create a package:

```
app/pages/audit/  
├─ __init__.py  
├─ page.py  
├─ form_builder.py
```

```
|— field_widgets.py
|— form_state.py
|— visibility_controller.py
|— defaults_controller.py
|— load_controller.py
|— save_controller.py
|— draft_controller.py
|— coach_controller.py
|— annotation_controller.py
|— machine_lookup_controller.py
|— compatibility_controller.py
|— result_messages.py
```

If moving from `app/pages/audit.py` to a package is too disruptive in one step, Codex can first create helper modules beside the existing file:

```
app/pages/audit_form_state.py
app/pages/audit_field_widgets.py
app/pages/audit_form_builder.py
```

Then later convert to a package.

Lowest-risk migration wins. The app does not need a cathedral today. It needs fewer wires dangling out of the toaster.

Implementation Order

Step 1: Extract form state

New file:

```
app/pages/audit_form_state.py
```

Responsibilities:

- Get current form values.
- Set form values.
- Mark baseline.
- Detect unsaved changes.
- Ignore generated Audit ID when appropriate.
- Ignore lookup-only fields when appropriate.

Functions/classes:

```
class AuditFormState:
    def current_values(self) -> dict[str, str]: ...
    def set_field_value(self, field: str, value: object) -> None: ...
    def set_values(self, values: Mapping[str, object]) -> None: ...
    def mark_baseline(self, reason: str = "") -> None: ...
    def has_unsaved_changes(self, ignored_fields: set[str] | None = None) ->
bool: ...
```

Step 2: Extract widget factory

New file:

```
app/pages/audit_field_widgets.py
```

Responsibilities:

- Create widgets from `AuditFieldSpec`.
- Apply validators.
- Apply dropdown values.
- Connect field-specific signals if needed.

Function:

```
def create_widget_for_field(spec: AuditFieldSpec, config, parent=None):
    ...
```

Step 3: Extract form builder

New file:

```
app/pages/audit_form_builder.py
```

Responsibilities:

- Build tabs from registry.
- Build group boxes.
- Add rows.
- Return field widget map and row metadata.

Function:


```
def build_audit_form_tabs(  
    field_specs,  
    widget_factory,  
    parent,  
) -> AuditFormBuildResult:  
    ...
```

Step 4: Extract draft controller

New file:

```
app/pages/audit_draft_controller.py
```

Responsibilities:

- Save draft.
- Restore draft.
- Discard draft.
- Offer draft recovery popup.
- Coordinate with form state.

Step 5: Extract visibility controller

New file:

```
app/pages/audit_visibility_controller.py
```

Responsibilities:

- Decide field visibility.
- Apply row visibility.
- Apply group visibility.
- Support Empty Only view.
- Explain hidden/non-applicable fields.

Step 6: Extract coach controller

New file:

```
app/pages/audit_coach_controller.py
```

Responsibilities:

- Refresh audit coach.
- Open coach target field.
- Mark field unknown.
- Create follow-up from coach.
- Apply manual completion override.
- Refresh after save/load.

Step 7: Extract load/save controllers

New files:

```
app/pages/audit_load_controller.py  
app/pages/audit_save_controller.py
```

Responsibilities:

Load controller:

- Load existing audit.
- Populate form.
- Duplicate audit.
- Generate new audit ID.
- Refresh audit selector.

Save controller:

- Gather values.
- Normalize.
- Validate.
- Save.
- Handle follow-up action.
- Handle compatibility update prompt.
- Emit events.
- Mark baseline clean after save.

Step 8: Extract annotation and lookup controllers

New files:

```
app/pages/audit_annotation_controller.py  
app/pages/audit_machine_lookup_controller.py  
app/pages/audit_compatibility_controller.py
```

Responsibilities:

- Field tag dialogs
 - Annotation target creation
 - Machine lookup
 - Capacity match handling
 - Compatibility row generation/update
-

Acceptance Criteria

Phase 4 is complete when:

- AuditPage is mostly orchestration.
 - Extracted modules have clear responsibilities.
 - User-facing behavior remains the same.
 - Save/load/draft/dirty tracking still works.
 - Empty Only mode still works.
 - Audit coach still works.
 - Manual override still works.
 - Cylinder fields still work.
 - Field visibility still works.
 - Tests pass.
-

Testing Requirements

Add or update tests for:

1. Form state detects changes correctly.
 2. Generated Audit ID does not falsely dirty a clean new form.
 3. Field widgets are generated correctly from schema.
 4. Cylinder widgets are generated correctly.
 5. Empty Only visibility still works.
 6. Hidden fields still save as where appropriate.
 7. Draft save/restore still works.
 8. Loading an audit still populates fields.
 9. Saving still updates workbook.
 10. Manual override still saves and refreshes coach.
 11. Compatibility update prompt still appears where expected.
-

Phase 5: Workbook Repository, Versioned Migrations, and Relationship Service

Phase Objective

Improve the data layer.

This phase separates audit domain logic from Excel workbook mechanics and adds a shared relationship service so different pages stop interpreting audit/compatibility rows differently.

This is the “make the data layer less haunted” phase. Very noble. Slightly tragic that it is necessary, but here we are.

Part A: Workbook Repository Separation

Expected Result

At the end of this part:

- Audit validation can run without opening Excel.
- Audit normalization can run without opening Excel.
- Workbook read/write logic is isolated.
- Legacy repair logic is isolated.
- Existing public functions still work through wrappers.

Proposed Files

```
core/audit/models.py
core/audit/normalizer.py
core/audit/validator.py
core/audit/repository.py
core/audit/workbook_repository.py
core/audit/legacy_repair.py
core/audit/save_service.py
```

Responsibilities

`models.py`

Defines audit data objects.

```
@dataclass(frozen=True)
class AuditEntry:
    audit_id: str
    values: dict[str, str]
    entry_type: str = "Audited"
```

normalizer.py

Handles:

- Applying defaults
- Clearing non-applicable fields
- Gripper model conversion
- Cylinder default behavior
- Part-present sensor defaults
- Changeover defaults
- Blank/NA conventions

validator.py

Handles:

- Required fields
- Important fields
- Numeric validation
- Dropdown validation
- Cylinder validation
- Semantic warnings
- Hybrid warnings

repository.py

Defines protocol:

```
class AuditRepository(Protocol):
    def list_audits(self) -> list[AuditEntry]: ...
    def get_audit(self, audit_id: str) -> AuditEntry | None: ...
    def save_audit(self, entry: AuditEntry) -> ToolResult: ...
```

workbook_repository.py

Implements the protocol using Excel.

Handles:

- Loading workbook
- Finding rows
- Writing rows
- Ensuring headers
- Detecting workbook locks
- Creating backups
- Saving workbook safely

`legacy_repair.py`

Handles old row formats and renamed columns.

Keeps legacy archaeology away from normal save logic. Software archaeology is cool until it is blocking you from adding two cylinder fields.

`save_service.py`

Coordinates:

- Normalize
- Validate
- Save
- Save robot-side data if needed
- Create follow-up action
- Emit result

Part B: Versioned Workbook Migrations

Expected Result

At the end of this part:

- Workbook schema updates are versioned.
- Migrations are safe to rerun.
- Workbook backups happen before migrations.
- Migration history is recorded.
- Cylinder and manual override columns are added through migrations.

Proposed Files

```
core/workbook_migrations/__init__.py
core/workbook_migrations/runner.py
```

```
core/workbook_migrations/metadata.py
core/workbook_migrations/v001_initial.py
core/workbook_migrations/v005_add_cylinders.py
core/workbook_migrations/v006_add_manual_completion_override.py
```

The exact version numbers can match the actual current migration state.

Metadata Storage

Recommended hidden sheet:

```
_EOAT_Metadata
```

Rows:

```
Key | Value | Updated At
```

Example:

```
workbook_schema_version | 0.1.6 | 2026-05-28T20:15:00
migration:v005_add_cylinders | applied | 2026-05-28T20:15:00
migration:v006_add_manual_completion_override | applied | 2026-05-28T20:16:00
```

Migration Rules

Every migration should be:

- Idempotent
- Backed up
- Logged
- Safe on existing workbooks
- Safe on demo workbooks
- Non-destructive unless explicitly confirmed

Cylinder Migration

Adds:

```
# of Cylinders
Cylinder Type
```

Also applies validations if workbook validation is supported.

Manual Override Migration

Adds:

```
Manual Completion Override
Manual Completion Override Timestamp
Ignored Empty Fields At Override
```

Optional:

```
Manual Completion Override Note
```

Part C: Audit Relationship Service

Expected Result

At the end of this part:

- Press View and Compatibility Entry agree.
- Audit Progress uses the same row classification logic.
- Workbook Health can detect relationship problems.
- Reports use consistent counts.
- Physical audit rows and compatibility rows are clearly separated.

Proposed File

```
core/audit/relationships.py
```

Proposed API

```
@dataclass(frozen=True)
class AuditRelationshipSummary:
    machine: str
    physical_audits: tuple[dict, ...]
    compatibility_entries: tuple[dict, ...]
    source_audits: tuple[dict, ...]
    compatible_tools: tuple[str, ...]
```



```

    warnings: tuple[str, ...]

def physical_audits_for_machine(project_root: str, machine: str) -> list[dict]:
    ...

def compatibility_entries_for_machine(project_root: str, machine: str) ->
list[dict]:
    ...

def compatibility_entries_for_source_audit(project_root: str, audit_id: str) ->
list[dict]:
    ...

def source_audit_for_compatibility_row(project_root: str, row: dict) -> dict |
None:
    ...

def relationship_summary_for_machine(project_root: str, machine: str) ->
AuditRelationshipSummary:
    ...

```

Rules

Physical audit row:

Entry Type = Audited

Compatibility row:

Entry Type = Compatible

Compatibility metadata fields:

Source Audit ID
Compatibility Source

Autofilled metadata fields should not be treated as required manual-entry fields.

Pages That Should Use This Service

Update:

Press View
Compatibility Entry
Audit Progress
Workbook Health
Reports

Acceptance Criteria

Phase 5 is complete when:

- Workbook persistence is separated from audit rules.
 - Workbook migrations are versioned.
 - Cylinder and manual override migrations exist.
 - Relationship service exists.
 - Press View and Compatibility counts are consistent.
 - Existing workbook save/load still works.
 - Demo project still works.
 - Tests pass.
-

Testing Requirements

Add tests for:

1. Normalizer works without Excel.
 2. Validator works without Excel.
 3. Workbook repository loads audits.
 4. Workbook repository saves audits.
 5. Migration runner is idempotent.
 6. Cylinder migration adds columns once.
 7. Manual override migration adds columns once.
 8. Existing data survives migration.
 9. Physical audit rows are classified correctly.
 10. Compatibility rows are classified correctly.
 11. Source audit relationships resolve correctly.
 12. Press View count matches relationship service.
 13. Compatibility Entry count matches relationship service.
-

Phase 6: Dashboard Routing, Event Bus Hardening, Tests, and Cleanup

Phase Objective

Finish the restructure by cleaning up dashboard routing, hardening events, improving tests, and removing transitional duplication.

This phase is where the app stops walking around with both the old and new architecture taped to its back like a software turtle.

Part A: Dashboard Route Registry

Problem

Dashboard deep-link/search result handling can become a long chain of action-specific conditionals.

Expected Result

Search and navigation actions use a route registry.

Proposed File

```
app/search_routes.py
```

Proposed Structure

```
class SearchRoute(Protocol):
    action: str

    def open(self, window, data: dict) -> bool:
        ...
```

Example registry:

```
SEARCH_ROUTES = {
    "open_audit": OpenAuditRoute(),
    "open_press": OpenPressRoute(),
```

```

    "open_note": OpenNoteRoute(),
    "open_open_item": OpenOpenItemRoute(),
    "open_validation": OpenValidationRoute(),
    "open_report": OpenReportRoute(),
    "open_photo": OpenPhotoRoute(),
}

```

`DashboardWindow.open_search_result()` should become small:

```

def open_search_result(self, result):
    data = result.to_dict() if hasattr(result, "to_dict") else dict(result)
    action = str(data.get("action") or "")
    route = SEARCH_ROUTES.get(action)
    if route is None:
        return False
    return route.open(self, data)

```

Acceptance Criteria

- Existing search/deep-link actions still work.
- `DashboardWindow` gets simpler.
- New routes can be added without editing a giant conditional chain.
- Unknown actions fail gracefully.

Part B: Event Bus Hardening

Problem

If one event subscriber fails, it should not prevent other subscribers from receiving the event.

Expected Result

Event handlers are isolated and failures are logged.

Implementation

Update event bus publish behavior:

```

def publish(self, event: AppEvent) -> None:
    handlers = self._handlers_for(event)
    for handler in handlers:

```

```
try:
    handler(event)
except Exception as exc:
    log_event_handler_error(event, handler, exc)
```

Add logging function:

```
def log_event_handler_error(event: AppEvent, handler, exc: Exception) -> None:
    ...
```

Log:

- event type
- source
- handler name
- exception type
- exception message
- timestamp

Acceptance Criteria

- One bad handler does not break other handlers.
- Failure is logged.
- Tests prove this.

Part C: Transitional Cleanup

Once the new systems are working, clean up old duplicate definitions.

Remove or convert:

- Old field lists now generated from registry
- Duplicate dropdown definitions
- Duplicate progress rules
- Duplicate workbook header lists
- Old compatibility count logic
- Old search action conditionals

Do this carefully. Do not remove compatibility wrappers until all callers are migrated.

Cleanup Rules

Before deleting old code:

1. Search for all imports.
2. Replace callers.
3. Run tests.
4. Delete old code.
5. Run tests again.
6. Run app smoke test.

Software cleanup without tests is just vandalism with confidence.

Part D: Testing and Tooling

Recommended `pyproject.toml`

Add:

```
[tool.ruff]
line-length = 120

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "SIM"]
ignore = []

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Recommended test folders

```
tests/core/audit/test_schema.py
tests/core/audit/test_completion.py
tests/core/audit/test_cylinders.py
tests/core/audit/test_manual_override.py
tests/core/audit/test_normalizer.py
tests/core/audit/test_validator.py
tests/core/audit/test_relationships.py
tests/core/workbook/test_migrations.py
tests/app/test_search_routes.py
tests/app/test_event_bus.py
tests/app/audit/test_audit_form_ui.py
```

Required Commands

Codex should run:

```
python -m pytest
python -m ruff check .
python scripts/repo_safety_audit.py --root .
```

If ruff is not added yet, skip that command and report why.

Final Acceptance Criteria for Phase 6

Phase 6 is complete when:

- Dashboard route registry exists.
 - Search result opening still works.
 - Event bus isolates handler failures.
 - Old duplicated field/progress/header logic is removed or marked as compatibility wrappers.
 - Tests cover new architecture.
 - Safety audit still passes.
 - Demo project still works.
 - App launches.
 - Audit save/load still works.
 - Cylinder fields still work.
 - Manual override still works.
 - Press View compatibility counts remain consistent.
 - Scheduled reports are not broken.
-

Final Six-Phase Delivery Checklist

Phase 1 Checklist

```
[ ] Create audit field registry
[ ] Add all existing fields
[ ] Add stable field IDs
[ ] Add registry helpers
[ ] Add schema tests
[ ] Preserve behavior
[ ] Run tests
```

Phase 2 Checklist

- ☐ Create completion engine
- ☐ Add excluded field policy
- ☐ Add optional group policy
- ☐ Add manual override support in core
- ☐ Refactor audit coach to consume completion results
- ☐ Add completion tests
- ☐ Run tests

Phase 3 Checklist

- ☐ Add # of Cylinders field
- ☐ Add Cylinder Type field
- ☐ Add Cylinder Details UI group
- ☐ Add cylinder workbook headers
- ☐ Add cylinder migration
- ☐ Add cylinder validation
- ☐ Add manual override button
- ☐ Add override confirmation popup
- ☐ Add override metadata fields
- ☐ Save/reload override
- ☐ Add tests
- ☐ Run tests

Phase 4 Checklist

- ☐ Extract form state
- ☐ Extract widget factory
- ☐ Extract form builder
- ☐ Extract draft controller
- ☐ Extract visibility controller
- ☐ Extract coach controller
- ☐ Extract load/save controllers
- ☐ Extract annotation/lookup/compatibility controllers
- ☐ Preserve behavior
- ☐ Run tests after each extraction

Phase 5 Checklist

- ☐ Add audit models
- ☐ Add normalizer

- ☐ Add validator
- ☐ Add workbook repository
- ☐ Add legacy repair module
- ☐ Add migration runner
- ☐ Add cylinder migration
- ☐ Add manual override migration
- ☐ Add relationship service
- ☐ Update Press View and compatibility consumers
- ☐ Add tests
- ☐ Run tests

Phase 6 Checklist

- ☐ Add dashboard search route registry
- ☐ Simplify dashboard search result handling
- ☐ Harden event bus
- ☐ Add event failure logging
- ☐ Remove old duplicate logic
- ☐ Add/strengthen tests
- ☐ Add ruff/pyproject if appropriate
- ☐ Run pytest
- ☐ Run safety audit
- ☐ Smoke test app launch

Final Expected Result

After all six phases, the EOAT Command Center should have:

- A central audit field schema
- Cleaner field additions
- Cleaner workbook header generation
- Safer workbook migrations
- Testable completion logic
- Truthful manual completion override
- Cylinder fields implemented correctly
- Smaller Audit page modules
- Consistent Press View and compatibility counts
- Safer event handling
- Cleaner dashboard routing
- Stronger tests
- Better long-term maintainability

Most importantly, adding the next audit field should feel boring.

Boring is the dream. Boring means the app behaves. Boring means nobody has to debug why adding `Cylinder Type` made `Press View` hallucinate seven compatibility rows and a fax machine.

The app should become boring in the best possible way: reliable, predictable, and hard to accidentally ruin.